Can We Play Football Outside?

Predicting play conditions from the weather with K-Nearest Neighbors

Item	Value
Input features	Temperature, Humidity, Rain
Target variable	Play_Outside (Yes / No)
Algorithm	K-Nearest Neighbors, K = 3
Dataset	weather_forecast_data.csv (2,500 rows)
Evaluation	Accuracy = 99.20 % on the test set
Web app	Streamlit (app.py)
Code	github.com/Unnaiiss/play-football-knn

1. Project Overview

Our project predicts **whether a person can go outside and play football** based on the weather. We use **Temperature**, **Humidity** and **Rain** as input features and **Play_Outside** as the target variable. We use the **K-Nearest Neighbors (KNN)** classification algorithm.

The dataset is divided into training and testing data. After preprocessing and feature scaling, KNN with K = 3 is applied. For a new weather condition, the model finds the three nearest weather observations and predicts whether playing outside is possible using majority voting. Finally, we evaluate the model using accuracy.

Project workflow

Step	What happens
1. Load data	Read the CSV file with pandas
2. Select features	Keep Temperature, Humidity, Rain
3. Preprocess	Remove missing/duplicate rows, encode Rain as 0/1
4. Create target	Build Play_Outside from a simple weather rule
5. Split	80 % training data, 20 % testing data
6. Scale	StandardScaler puts all features on the same scale
7. Train	KNN with K = 3
8. Evaluate	Accuracy, confusion matrix, precision and recall
9. Predict	Answer Yes/No for a new day's weather
10. Deploy	Interactive Streamlit web app

Project files

File	Purpose
weather_forecast_data.csv	The dataset (2,500 weather observations)
Play_Outside_KNN.ipynb	Jupyter notebook: full analysis step by step
app.py	Streamlit web app
requirements.txt	Libraries Streamlit Cloud must install

2. How K-Nearest Neighbors Works

KNN is one of the simplest machine-learning algorithms. It is based on one idea: **similar weather gives similar results**. If the days most like today were good for football, today probably is too.

- **Store**: KNN keeps all training observations in memory. It does not build equations; this is why it is called a *lazy learner*.
- **Measure distance**: for a new day, it computes the distance to every stored observation (Euclidean distance, i.e. a straight line between points).
- **Pick K neighbours**: it selects the K = 3 closest observations.
- **Vote**: each neighbour votes with its label. With 2 or 3 "Yes" votes the prediction is Yes; otherwise it is No.

Euclidean distance between two days A and B:

```
distance = sqrt( (TempA - TempB)^2 + (HumA - HumB)^2 + (RainA - RainB)^2 )
```

Why K = 3? K = 1 copies a single neighbour and reacts to noise. A very large K blurs the boundary between classes. An **odd** number avoids a tie in a Yes/No vote, and 3 is small enough to keep the decision local.

3. The Dataset

The file `weather_forecast_data.csv` has 2,500 rows and 6 columns: Temperature, Humidity, Wind_Speed, Cloud_Cover, Pressure and Rain. Our project uses only three of them.

Feature	Type	Min	Max	Mean	Used?
Temperature (°C)	Number	10.0	35.0	22.6	Yes
Humidity (%)	Number	30.0	100.0	64.3	Yes
Rain	Text: rain / no rain	-	-	-	Yes
Wind, Cloud, Pressure	Number	-	-	-	No

Creating the target variable Play_Outside

The dataset has no Play_Outside column, so we create one with a common-sense rule. A day is good for football (**1 = Yes**) only when **all** conditions are true; otherwise it is **0 = No**.

Condition	Reason
No rain	A wet field is slippery and unsafe
15 °C <= Temperature <= 32 °C	Not too cold and not too hot
Humidity <= 85 %	Very humid air makes running exhausting

Result: **1,240 Yes** and **1,260 No** days, an almost even split, which is good for training.

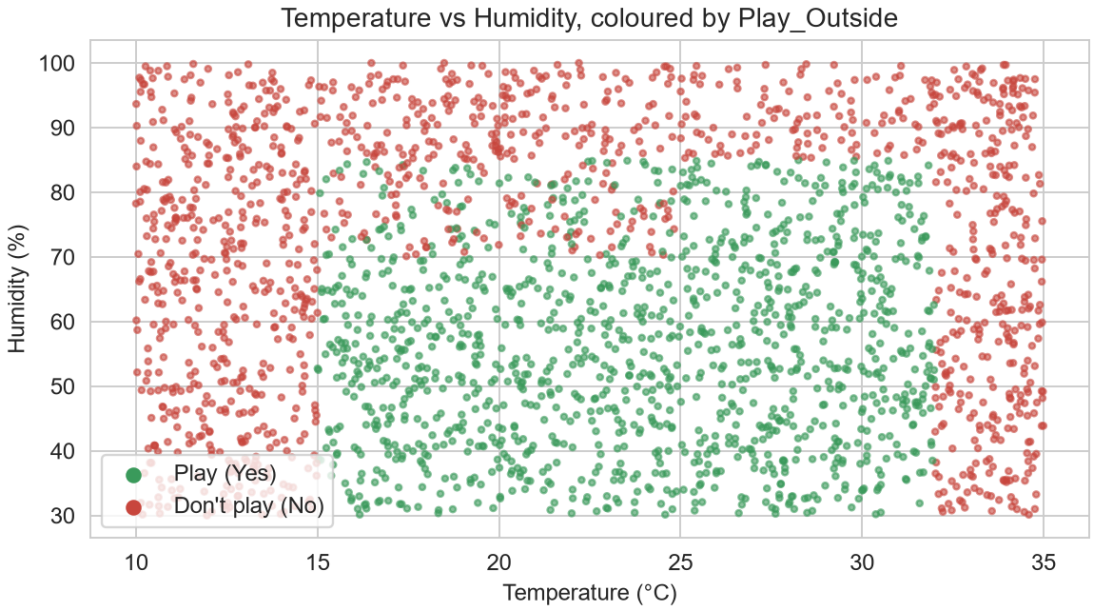


Figure 1. Green days are good for football. They sit in the middle temperature band with lower humidity. Rainy days are always No.

4. Code Explained Step by Step

This section follows the notebook `Play_Outside_KNN.ipynb` cell by cell.

Step 1 - Import libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

- **pandas** reads and handles tables of data.
- **matplotlib / seaborn** draw the charts.
- **scikit-learn** provides the split, the scaler, the KNN model and the accuracy metric.

Step 2 - Load the data and keep the needed columns

```
df = pd.read_csv("weather_forecast_data.csv")
df = df[["Temperature", "Humidity", "Rain"]].copy()
```

`read_csv` loads the file into a table called a DataFrame. We then keep only the three columns the project uses.

Step 3 - Preprocessing

```
df = df.dropna()            # remove rows with missing values
df = df.drop_duplicates()   # remove repeated rows
df["Rain"] = df["Rain"].map({"rain": 1, "no rain": 0})
```

KNN works with **distances**, so every value must be a number. The text values of Rain are converted to numbers: *rain* becomes 1 and *no rain* becomes 0. This is called **encoding**.

Step 4 - Create the target

```
df["Play_Outside"] = (
    (df["Rain"] == 0)
    & (df["Temperature"].between(15, 32))
    & (df["Humidity"] <= 85)
).astype(int)
```

Each condition gives True/False for every row. The & operator means "and", so all three must be True. `astype(int)` turns True/False into 1/0.

Step 5 - Split features and target, then train/test

```
X = df[["Temperature", "Humidity", "Rain"]]   # inputs
y = df["Play_Outside"]                        # answer to learn

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- **test_size=0.2**: 20 % of rows (500) are hidden for testing; 80 % (2,000) are used for training.
- **random_state=42**: makes the random split the same every time, so results can be repeated.
- **stratify=y**: keeps the same Yes/No ratio in both parts.

Step 6 - Feature scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # learn mean/std from training data
X_test_scaled  = scaler.transform(X_test)        # apply the same scaling to test data
```

Humidity ranges from 30 to 100 but Rain is only 0 or 1. Without scaling, Humidity would dominate the distance. StandardScaler changes each feature to **mean 0 and standard deviation 1**:

```
scaled value = (value - mean) / standard deviation
```

Important: the scaler is *fitted* on the training data only. If it also saw the test data, information would leak from the test set and the accuracy would look better than it really is.

Step 7 - Train KNN with K = 3

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)
```

`n_neighbors=3` sets K. For KNN, `fit` simply stores the training data, which is used later to find neighbours.

Step 8 - Evaluate with accuracy

```
y_pred = knn.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

The model predicts every test row and compares its answers with the real ones.

```
accuracy = correct predictions / total predictions
```

Step 9 - Predict a new day

```
new_day = pd.DataFrame([[24, 55, 0]], columns=["Temperature", "Humidity", "Rain"])
new_day_scaled = scaler.transform(new_day)            # same scaler!
distances, indices = knn.kneighbors(new_day_scaled)   # the 3 nearest days
prediction = knn.predict(new_day_scaled)[0]           # majority vote: 1 = Yes
```

A new day must be scaled with the **same** scaler before prediction. `kneighbors` returns the 3 closest training days so we can show how the vote went.

5. Results

Metric	Value
Accuracy	99.20 %
Correct predictions	496 of 500
Precision (Yes)	0.988
Recall (Yes)	0.996
F1-score (Yes)	0.992

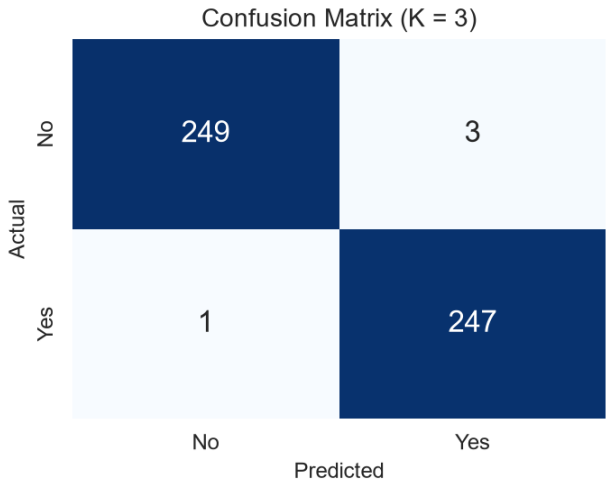


Figure 2. Confusion matrix. The diagonal (249 + 247) are correct; only 4 test days were misclassified.

- **Precision**: of the days the model said Yes, how many really were Yes.
- **Recall**: of the real Yes days, how many the model found.
- **Mistakes** happen on days right on a boundary, e.g. 31.9 °C vs 32.1 °C, where neighbours fall on both sides of the rule.

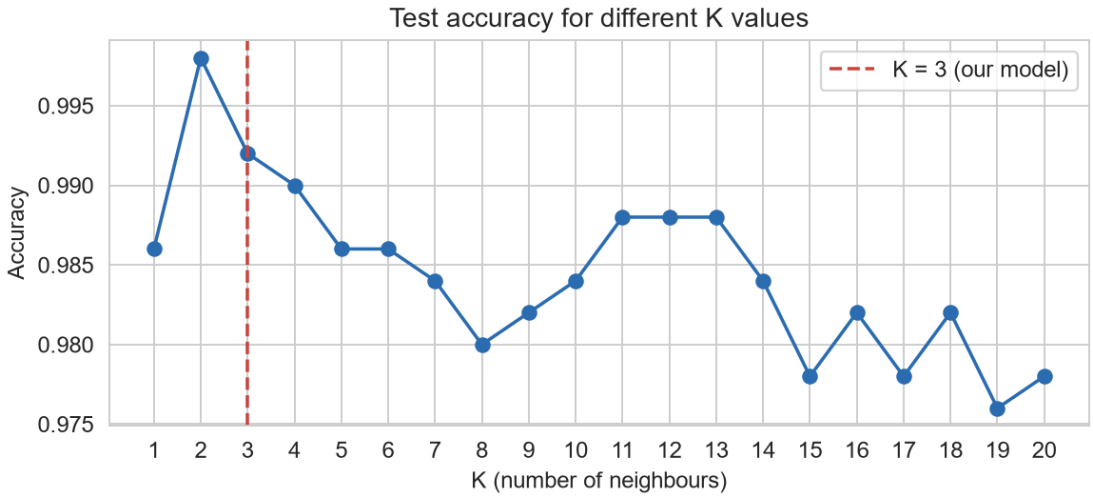


Figure 3. Accuracy for K = 1 to 20. K = 3 performs well, which supports our choice.

Example predictions

Temperature	Humidity	Rain	Neighbour votes	Prediction	Note
24 °C	55 %	No	3 Yes / 0 No	YES	Nice day
22 °C	70 %	Yes	0 Yes / 3 No	NO	Raining
34 °C	60 %	No	0 Yes / 3 No	NO	Too hot

6. Streamlit Web App

The file `app.py` turns the model into an interactive web page using **Streamlit**, a Python library that builds web apps without HTML or JavaScript.

```
@st.cache_resource
def train_model():
    ...  # same steps as the notebook: load, encode, split, scale, fit KNN
    return df, X_train, y_train, scaler, knn, accuracy

temperature = st.sidebar.slider("Temperature (°C)", 10.0, 35.0, 24.0, 0.5)
humidity    = st.sidebar.slider("Humidity (%)", 30.0, 100.0, 55.0, 1.0)
rain        = st.sidebar.radio("Is it raining?", ["No", "Yes"])

prediction = knn.predict(scaler.transform(new_day))[0]
if prediction == 1:
    st.success("YES - go play football!")
else:
    st.error("NO - better stay inside.")
```

- **@st.cache_resource** trains the model once and reuses it, so the app stays fast.
- **Sliders and a radio button** in the sidebar collect the weather.
- Every time an input changes, Streamlit reruns the script and shows the new prediction, the 3 nearest neighbours with their votes, and a chart marking today's weather.

Running and deploying

```
# run locally
pip install -r requirements.txt
streamlit run app.py          # opens http://localhost:8501

# deploy: push to GitHub, then on share.streamlit.io choose
#   repo: Unnaiiss/play-football-knn   branch: main   file: app.py
```

7. Conclusion

- We built a model that predicts whether a person can play football outside using Temperature, Humidity and Rain.
- The data was cleaned, Rain was encoded as 0/1, and the data was split 80/20 into training and testing sets.
- Feature scaling gave every feature a fair share of the distance.
- KNN with K = 3 reached **99.20 % accuracy** on unseen test data.
- The model is available as an interactive Streamlit web app.

Limitations and future work

- Play_Outside was created from a rule, not collected from real players. That is why accuracy is so high: KNN is learning a clear rule. Real survey data would be noisier and more realistic.
- Wind speed and cloud cover could also affect play and could be added as features.
- K could be chosen with cross-validation instead of a single train/test split.